

8IAR403 : Apprentissage automatique pour la science des données

Devoir #2 Entraînement et métriques de performance de l'apprentissage

Travail individuel

Date de remise : le lundi 16 mars 2026

1. But

- Poursuite de la validation de la méthodologie de conduite de projets en Machine Learning vue en cours sur une étude de cas de vente en ligne;
- Se familiariser davantage avec l'étape d'entraînement et la mesure de performance d'un modèle appris de cette méthodologie;
- Entraînement de plusieurs classeurs : binaire, multi-classes.
- Validation, réglage des hyperparamètres et test

2. Étude de cas

Le contexte métier utilisé sera celui que vous avez étudié durant le devoir#1. Le but était de produire une TBA nettoyée et prête à être utilisée dans l'étape de modélisation (apprentissage machine) à savoir l'entraînement d'un algorithme d'apprentissage supervisé afin d'apprendre les profils qui discriminent la vente en ligne et plus particulièrement, classer les revenus que vont générer de nouveaux clients du site de e-commerce. Par conséquent, chercher le profil du bon client. Par ailleurs, cette phase de préparation du devoir#1, a permis d'avoir une TBA qui sera le réservoir d'entraînement (p.e. 80% des 10000 observations du dataset initial) et un jeu de Test (20% restant) qui doit être aussi mis dans une même configuration avant d'être exploité pour les tests.

Le but visé dans ce devoir#2 est d'entraîner, selon la démarche « recommandée » vue en cours, l'algorithme `DecisionTreeClassifier` de `scikit-learn` pour produire plusieurs classeurs à partir de différents échantillons de différentes tailles du jeu d'entraînement. Ces différents modèles seront tous testés avec le même jeu de test qui a été mis de côté durant la phase de préparation.

2.1 Entraînement d'un classifieur binaire (10 points)

Dans un premier temps, on souhaite créer un **classifieur binaire** dont la variable cible est le revenu. Il s'agit de prédire si un client est susceptible ou non de générer plus de revenus que la **moyenne**.

- 2.1.1 Vous devez écrire une fonction pour remplacer cette variable « revenu » par une variable binaire ayant deux valeurs. Par exemple, 1 pour un revenu supérieur à la moyenne et 0 sinon.
- 2.1.2 Échantillonnage aléatoire en utilisant la méthode « resample » de scikit-learn. Chaque échantillon extrait aléatoirement du jeu d'entraînement, sera utilisé pour entraîner l'algorithme selon la démarche « recommandée » vue en cours. Par ailleurs, étant donné la lourdeur de certaines opérations d'entraînement, vous devez envisager des tailles progressives pour les échantillons, par exemple, 2000, 4000, 8000 observations. Pour ce qui est des données, vous devez faire des échantillons selon les 3 scénarios suivants:
 1. Sans le PIB et sans la Population : uniquement le dataset Customer
 2. Sans le PIB : jointure des deux datasets Customer et CountryPopulation
 3. Avec le PIB : jointure des trois : Customer, CountryPopulation et CountryGDP
- 2.1.3 Entraîner et tester par la validation croisée pour les deux valeurs de k suivantes : 3 et 10.
- 2.1.4 Mesurer les métriques de précision, rappel et F1-Mesure.
- 2.1.5 Optimiser les hyper-paramètres en utilisant la recherche par quadrillage –GridSearchCV-. Étant donné, la lourdeur en temps de cette opération, prenez juste 2 hyper-paramètres : la profondeur de l'arbre et le nombre minimal d'observations pour une segmentation d'un nœud. Il faudrait valider avec plusieurs plages de valeurs pour chaque hyper-paramètre. Pour le nombre de blocs (k) prenez la meilleure valeur de k trouvée en 2.1.3.
- 2.1.6 Tester le **meilleur modèle** obtenu à l'étape 2.1.5 avec le « vrai » jeu de test et mesurer les différentes métriques de précision, rappel et F1-Mesure.
- 2.1.7 Représenter graphiquement et à l'aide de tableaux l'ensemble des résultats des métriques de performance selon les 3 scénarios de datasets et ainsi que les différents facteurs mis en jeu : taille des échantillons, nombre de passes (bloc) k, la plage de valeurs des hyper-paramètres et le temps d'entraînement (réponse) pour l'obtention des modèles en ajoutant le temps du GridSearchCV. Le but est de justifier le choix du modèle.
- 2.1.8 Analyser et interprétez les résultats obtenus.

2.2. Entraînement d'un classifieur multi-classes (10 points)

Créer un classifieur capable de reconnaître différentes catégories de revenus qui peuvent être générées. Par exemple, nous nous concentrerons sur les trois classes suivantes : **revenu-bas**, **revenu-moyen**, **revenu-élevé**. Pour cela, vous devez discrétiser la variable cible revenu selon une méthode de discrétisation de votre choix ou selon la méthode suivante : Transformer la distribution de la variable revenu en une distribution normale en utilisant, par exemple, l'utilitaire **PowerTransformer()** de scikit-learn et, par la suite appliquer la règle suivante en considérant la moyenne μ et l'écart type σ de la nouvelle distribution de la variable revenu :

- Revenu-moyen : valeur de revenu $\in [\mu - \sigma, \mu + \sigma]$
- Revenu bas : valeur de revenu $< \mu - \sigma$
- Revenu élevé : valeur de revenu $> \mu + \sigma$

- 2.1.1 Vous devez écrire une fonction pour remplacer cette variable « revenu » par une variable multi-classes selon les trois catégories de revenu;
- 2.1.2 Quel est le meilleur classeur pouvant discriminer les 3 revenus et justifiez votre réponse?
- 2.1.3 Comparer les résultats obtenus avec ceux du classeur binaire ?

3 Livrable

- 3.1 Fournir un ou plusieurs fichiers **ipynb** comprenant les étapes demandées. Je vous recommande de travailler avec jupyterlab (colab, anaconda,...)
- 3.2 **Pour me faciliter la correction, vous devez afficher le résultat d'exécution de chaque cellule du notebook. Cela m'évitera de re-exécuter votre code, à chaque fois.**
- 3.3 Commenter chacune des étapes en utilisant Markdown et en faisant référence aux questions posées. Même s'il n'y a pas de réponse à une question, il faut l'indiquer dans une cellule du notebook pour me faciliter la correction. Les réponses doivent respecter l'ordre des questions.
- 3.4 Indiquer dans l'introduction du fichier **ipynb**, **votre nom et prénom** et si vous avez ajouté de nouvelles fonctionnalités non-demandées. Il faut les commenter.
- 3.5 **Transmettez un fichier compressé en .zip. Veuillez le déposer dans le dossier « dépôt » prévu sur le site moodle du cours, dans la section « devoir 2 ». A Éviter de déposer sur d'autre site de téléchargement ! AUCUN RENDU PAR COURRIEL NE SERA ACCEPTÉ**
- 3.6 Le nommage du rendu compressé doit être comme suit: nom_prénom_tp2_h2026.zip

Critères de correction

Critères	Points
Classeur binaire	
Q2.1.1	/1
Q.2.1.2	/1
Q.2.1.3-2.1.6 (1 point par Question)	/4
Q2.1.7	/2
Q2.1.8	/2
Classeur multi-classes	
Q2.1.2	/4
Q2.2.2	/4
Q2.2.3	/2

Total : /20

Bonne continuation !